Case Study Report

System Design (22CS024)

BACHELOR OF ENGINEERING

In

Computer Science and Engineering

Submitted By :-

Abhay Pratap Singh- 2310990091

Amanjot Singh- 2310990106

Ansuman- 2310990120

Dhruv Sharma-2310990155

Submitted to:-

Mr. Pankaj

Group-2

Sem-5th Year 3rd

CHITKARA UNIVERSITY,

PUNJAB

CHANDIGARH-PATIALA

NATIONAL HIGHWAY RAJPURA

(PATIALA) PUNJAB-140401

(INDIA)

INDEX

This index provides an overview of the document structure, adhering strictly to the required section titles and estimated page ranges for a comprehensive 25-30 page technical report.

Sr. No.	Content	Page No. (Estimated)
I	**Core Analysis and Requirements**	**1-5**
1.	Problem Statement	1
2.	Objectives	2
3.	Functional and Non Functional Requirements	3-5
II	**System Design and Architecture**	**6-22**
4.	High-Level Design: System Context Diagram including Major Components (API Gateway, Services, Databases, External Integrations, etc.) including the Data Flow Overview	6-8
5.	Low-Level Design	9-17
6.	Database Design	18-19
7.	Network System Design	20-22
III	**Conclusion and Future Roadmap**	**23-25**
8.	Future Improvements	23-25

1. PROBLEM STATEMENT (Page 1)

Small to medium enterprises often rely on fragmented, manual, or legacy systems (like spreadsheets) to track inventory. This leads to critical operational issues that inhibit growth and reduce profitability:

1. **Stock Discrepancies and Data Lag:** Manual entry results in frequent errors (e.g., transposing digits, double counting) between physical and recorded stock counts. Furthermore, data is not updated in real-time, creating a data lag that makes fulfillment planning inaccurate.
2. **Delayed Visibility Across Locations:** Lack of real-time, consolidated data on stock levels across multiple warehouses or stores makes order fulfillment difficult. Personnel cannot quickly confirm stock availability in alternate locations for immediate transfer or drop-shipping, leading to missed sales opportunities or unnecessary cancellation of customer orders.
3. **Inefficient and Reactive Ordering:** Without automated, threshold-based alerts, the purchasing department operates reactively. This results in the "bullwhip effect"—either running into costly **stockouts** (leading to lost revenue) or **overstocking** (leading to high carrying costs and potential obsolescence).
4. **Lack of Auditability and Accountability:** In the absence of a proper logging system, there is no easy way to track *who* moved *what* item, *when*, and *why*. This gap in the audit trail severely complicates loss prevention efforts, inventory variance analysis, and user accountability.

2. OBJECTIVES (Page 2)

The IMS project was designed to solve the above problems with the following clear and measurable objectives:

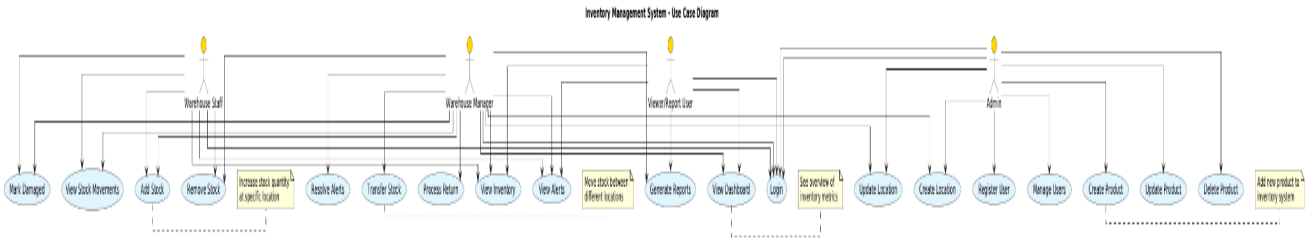
Category	Objective	Description	Success Metric
Data Integrity	**Single Source of Truth**	Centralize all product, location, and stock data in a secure, unified MongoDB database, eliminating data silos.	$\le 0.5\%$ stock discrepancy rate between physical and recorded counts post-launch.
Real-Time Visibility	**Instant Stock Levels**	Provide immediate, up-to-date total stock and location-specific quantities via a dedicated API endpoint.	Dashboard loading time for core stock metrics ≤ 100 ms.
Automation	**Automated Alerting**	Implement threshold-based alerts for low stock, out-of-stock, and damaged inventory.	System generates alerts within 5 seconds of a threshold breach.
Accountability	**Complete Audit Trail**	Log every stock movement (add, remove, transfer, damage, return) with user attribution and reason.	100% of all stock modifications are recorded in the stockMovements sub-schema.
Usability	**Intuitive Interface**	Develop a responsive, user-friendly React	User task completion time for "Add Stock"

		application for quick navigation and task execution across all common devices (desktop, tablet, mobile).	operation ≤ 15 seconds.

3. FUNCTIONAL AND NON FUNCTIONAL REQUIREMENTS (Pages 3-5)

3.1 Functional Requirements (FR) - Detailed Breakdown

System Design



Requirement ID	Requirement	Description
FR-AUTH-001	User Management (Auth)	Secure registration, login (JWT), and user profile retrieval via dedicated secure endpoints.
FR-PROD-002	Product CRUD	Complete set of operations (Create, Read, Update, Delete) for product masters, including data fields like **ProductName**, **ProductBarcode** (unique index), **ProductCategory**, and **globalMinStock**.
FR-LOC-003	Location Management	Define, edit, and activate multiple storage locations (e.g., "Warehouse A", "Retail Store B"). Locations must be uniquely identifiable.
FR-STOCK-004	Stock Inbound	API endpoint to increase stock quantity for a specific product at a specified location, requiring a **reason** and **reference ID**

		(e.g., PO Number).
FR-STOCK-005	Stock Outbound	API endpoint to decrease stock quantity, requiring validation to ensure quantity does not exceed available stock at the specified location.
FR-STOCK-006	Stock Transfer	Dedicated endpoint to facilitate stock movement between two locations, performing an atomic decrease at the source and increase at the destination.
FR-STOCK-007	Damage/Loss Marking	Endpoint to mark a specific quantity of stock as "damaged" or "lost" at a location, adjusting the available quantity and adding an 'adjustment' movement entry.
FR-ALERT-008	Alert Management	Dedicated pages/components to view, filter (by severity, location), mark as read, and resolve automated stock alerts (StockAlert documents).
FR-DASH-009	Dashboard Overview	Display key performance indicators (KPIs) like **Total Products**, **Low Stock Products**, **Unread Alerts**, and a feed of the **Latest Stock Movements**.
FR-AUDIT-010	Stock Movement History	Dedicated view to display the complete, paginated audit trail (stockMovements sub-schema) for any given product.

3.2 Non-Functional Requirements (NFR) - Security and Performance

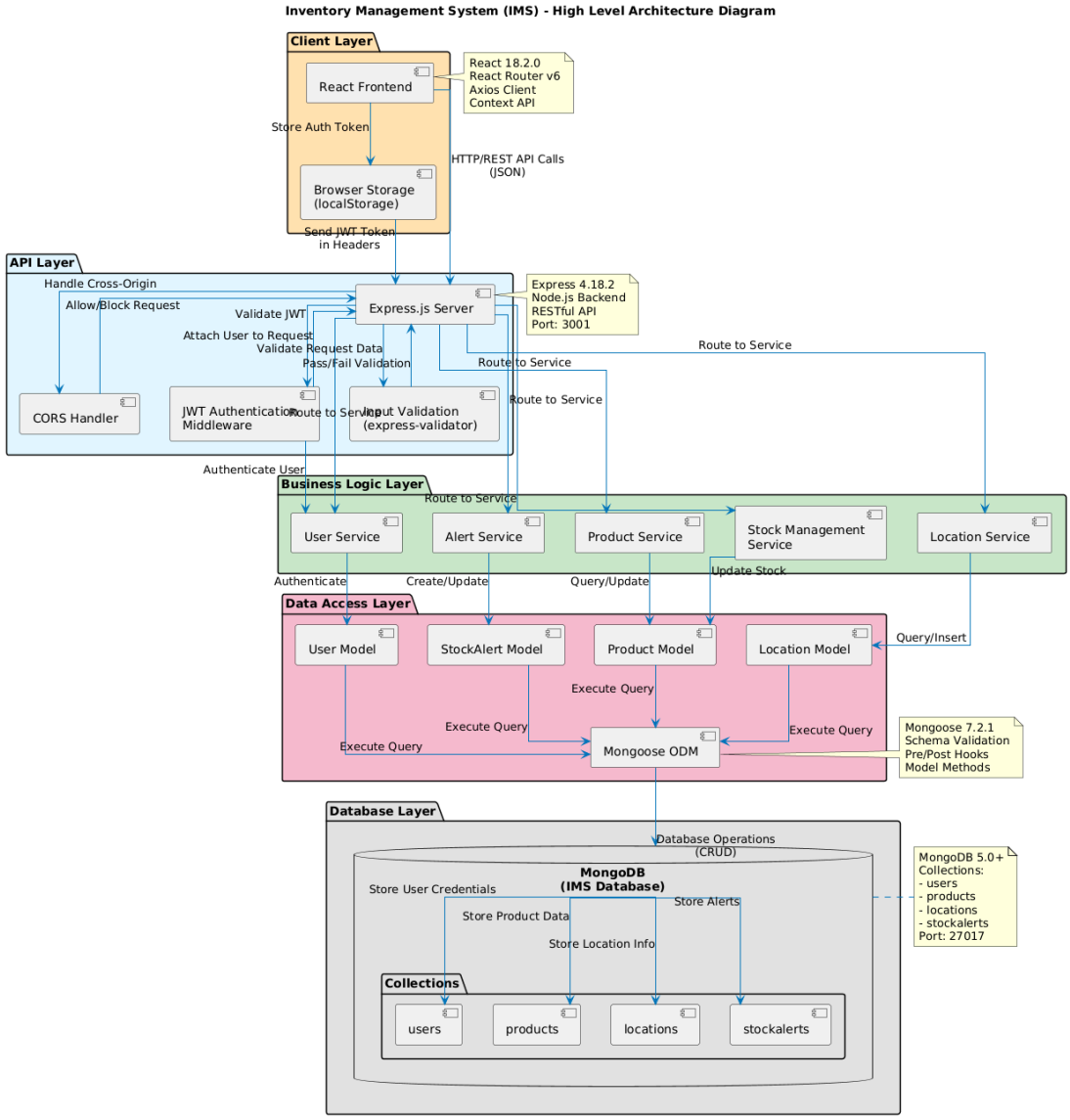
Requirement ID	Requirement	Description	Metric/Standard
NFR-SEC-001	Security (Authentication)	Secure authentication for all protected resources. Passwords must be non-reversible.	**JWT** (signed with strong secret), **bcrypt** (salt rounds: ≥ 10).
NFR-SEC-002	Authorization	All product and location data must be scoped to the creating user.	**User-scoped data access** enforced via fetchuser middleware and Mongoose query filters ({createdBy: req.userId}).
NFR-PERF-003	API Performance	Fast response times for stock updates and dashboard loads.	API Latency: **p95** (95th percentile) for read operations ≤ 100 ms.
NFR-PERF-004	Database Indexing	Optimal use of database indexes to support frequent search and lookup operations.	Unique index on **ProductBarcode**. Compound index on {createdBy, totalStock}.
NFR-SCAL-005	Scalability	Architecture must allow easy vertical/horizontal scaling of the API and database tiers to handle traffic growth.	MERN Stack architecture is inherently horizontally scalable via Load Balancing and MongoDB Replica Sets.
NFR-REL-006	Reliability (Atomicity)	Stock updates must be atomic and error-resistant,	Data integrity via **Mongoose Pre-save**

		ensuring quantity and audit trails are updated simultaneously.	**middleware** and single document updates.
NFR-MAINT-007	Maintainability	Codebase must be modular, well-commented, and logically separated.	Strict MVC pattern adherence in the backend; Component-based structure in the React frontend.

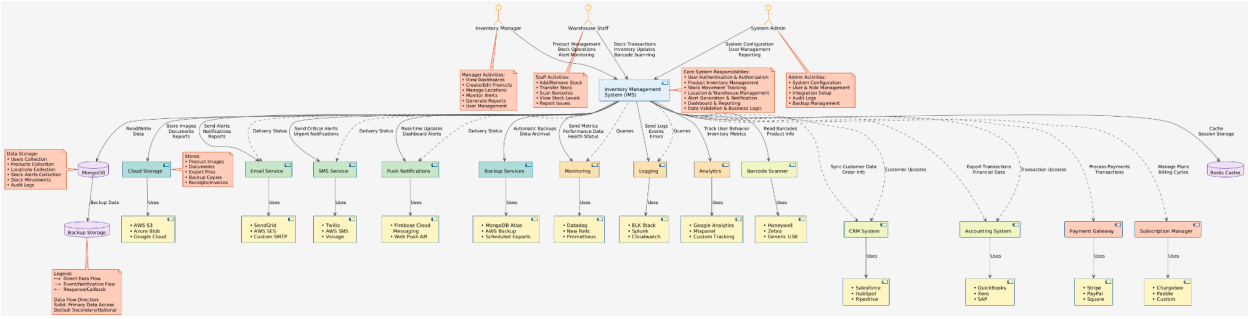
4. HIGH-LEVEL DESIGN: System Context Diagram including Major Components (API Gateway, Services, Databases, External Integrations, etc.) including the Data Flow Overview (Pages 6-8)

The Inventory Management System adheres to a three-tier MERN architecture, ensuring clear separation of responsibilities for scaling and maintenance.

Component Diagram:



4.1 System Context Diagram - Architectural Tiers



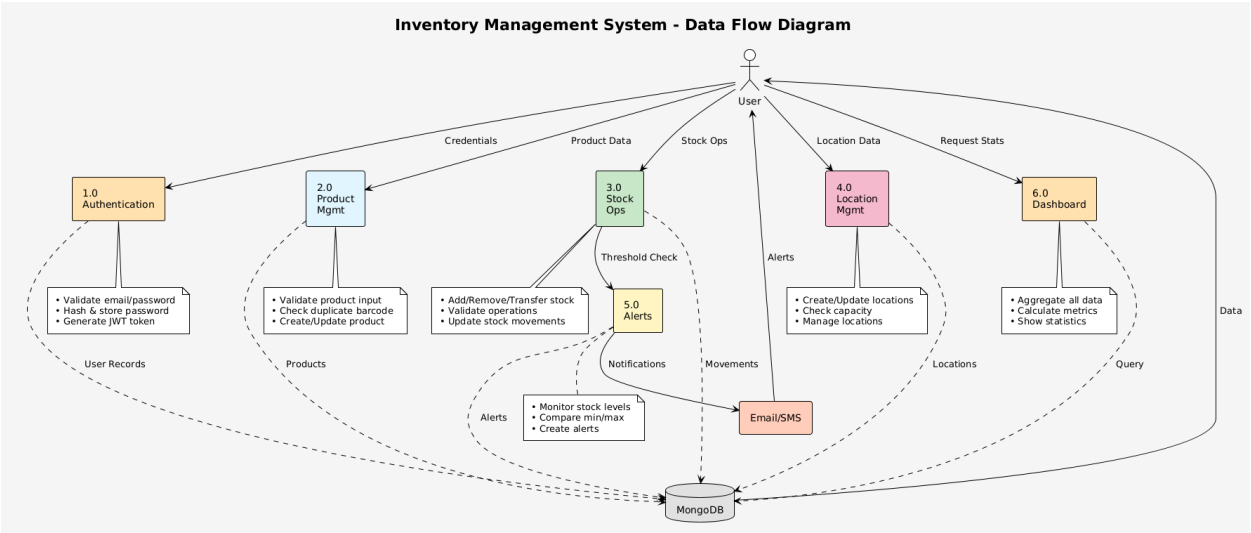
The system is compartmentalized into the Presentation, Application, and Data layers.

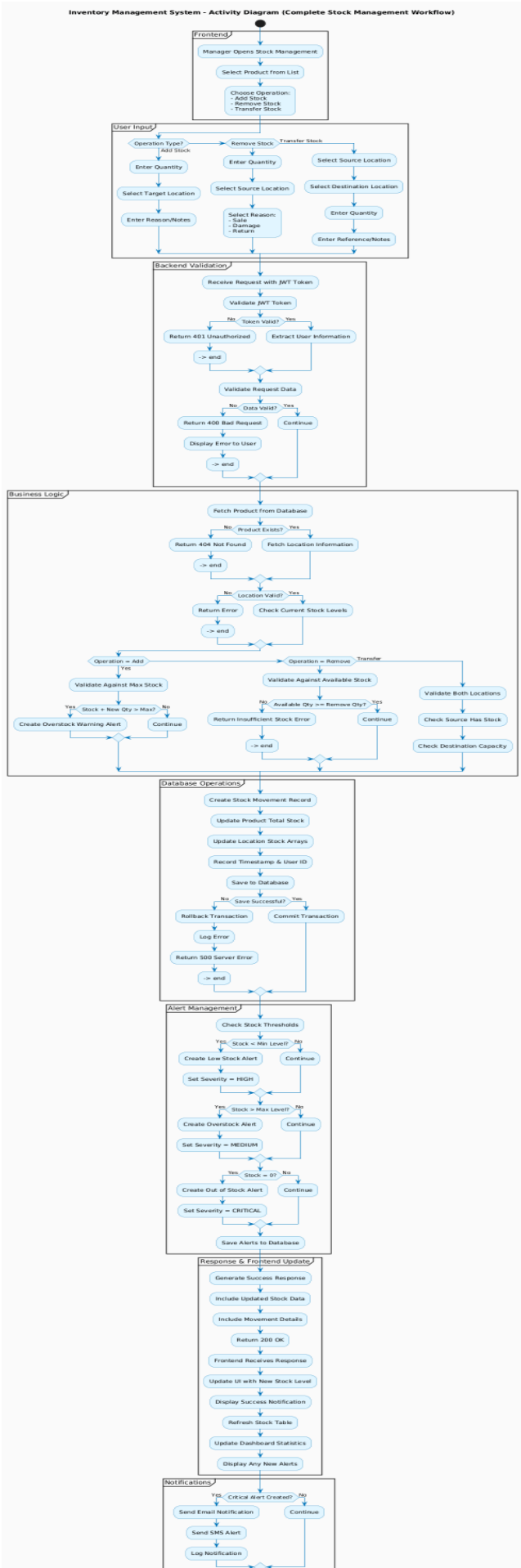
- **Presentation Layer (Web Client - React):** The user interface. It is responsible solely for rendering, user input, and state management. It communicates with the Application Layer exclusively via RESTful API calls using the **Axios** library.
- **Application Layer (API Gateway - Express/Node.js):** This acts as the centralized business logic server. It handles incoming requests, validates tokens (fetchuser middleware), executes complex business logic (e.g., calculating stock levels, generating alerts), and performs database interactions via Mongoose Models.
- **Data Layer (Data Store - MongoDB):** The persistent data storage. It holds the core master data and the audit trails, optimized for high-read performance through embedded documents.
- **External Service (Notification Engine - Future):** Represents a future service integration (e.g., SendGrid, Twilio) for sending automated out-of-band alerts via email or SMS.

**

4.2 Core Data Flow: The Stock Update Transaction (Example: POST /stock/add/:productId)

Activity Diagram:




Inventory Management System - Activity Diagram (Complete Stock Management Workflow)
Frontend
Manager Opens Stock Management
Select Product from List
Choose Operation:
- Add Stock
- Remove Stock
- Transfer Stock
User Input
Operation Type?
Add Stock
Remove Stock
Transfer Stock
Enter Quantity
Select Target Location
Enter Reason/Notes
Enter Quantity
Select Source Location
Select Reason:
- Sale
- Damage
- Return
Select Source Location
Select Destination Location
Enter Quantity
Enter Reference/Notes
Backend Validation
Receive Request with JWT Token
Validate JWT Token
Token Valid?
No
Yes
Return 401 Unauthorized
Extract User Information
-> end
Validate Request Data
Data Valid?
No
Yes
Return 400 Bad Request
Continue
Display Error to User
-> end
Business Logic
Fetch Product from Database
Product Exists?
No
Yes
Return 404 Not Found
Fetch Location Information
-> end
Location Valid?
No
Yes
Return Error
Check Current Stock Levels
-> end
Operation = Add
Operation = Remove
Transfer
Yes
Validate Against Max Stock
Stock + New Qty > Max?
Yes
No
Create Overstock Warning Alert
Continue
Validate Against Available Stock
Available Qty >= Remove Qty?
No
Yes
Return Insufficient Stock Error
Continue
-> end
Validate Both Locations
Check Source Has Stock
Check Destination Capacity
Database Operations
Create Stock Movement Record
Update Product Total Stock
Update Location Stock Arrays
Record Timestamp & User ID
Save to Database
Save Successful?
No
Yes
Rollback Transaction
Commit Transaction
Log Error
Return 500 Server Error
-> end
Alert Management
Check Stock Thresholds
Stock < Min Level?
Yes
No
Create Low Stock Alert
Continue
Set Severity = HIGH
Stock > Max Level?
Yes
No
Create Overstock Alert
Continue
Set Severity = MEDIUM
Stock = 0?
Yes
No
Create Out of Stock Alert
Continue
Set Severity = CRITICAL
Save Alerts to Database
Response & Frontend Update
Generate Success Response
Include Updated Stock Data
Include Movement Details
Return 200 OK
Frontend Receives Response
Update UI with New Stock Level
Display Success Notification
Refresh Stock Table
Update Dashboard Statistics
Display Any New Alerts
Notifications
Critical Alert Created?
Yes
No
Send Email Notification
Continue
Send SMS Alert
Log Notification

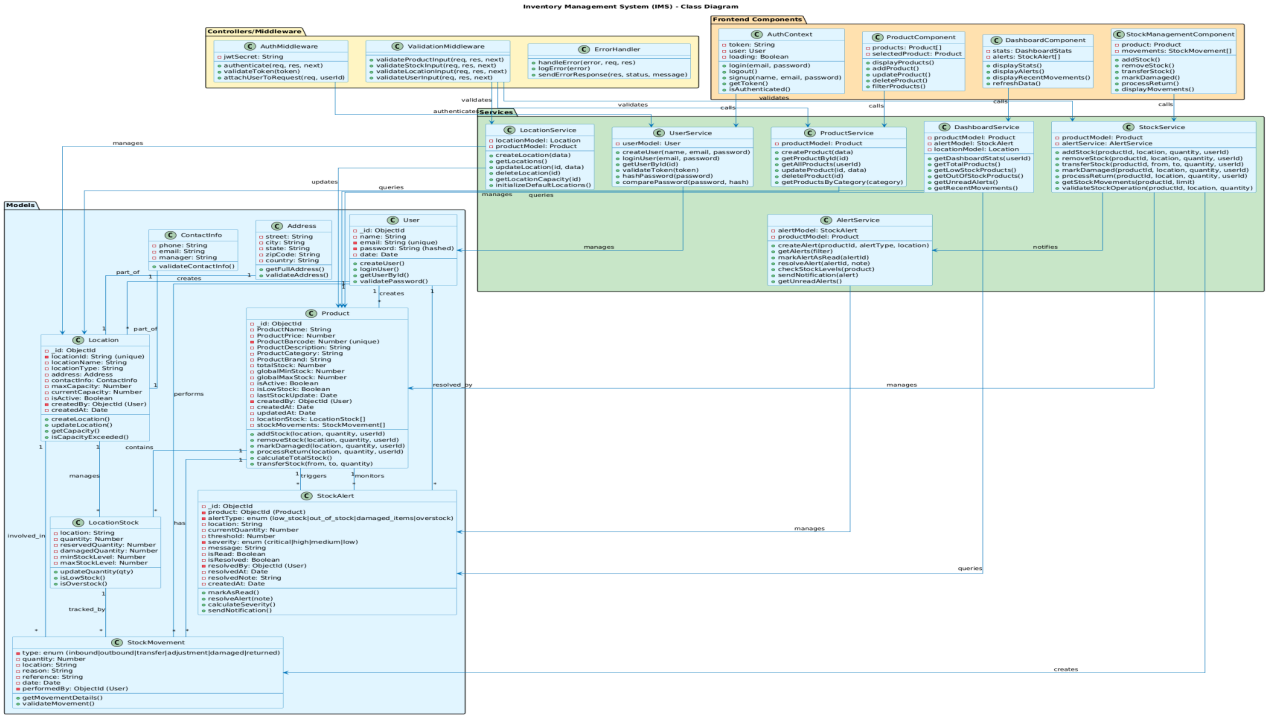
The Stock Update Transaction is the most critical flow, designed for atomicity and immediate consistency, performed on a single Products document.

Step	Component	Action Description	Integrity Mechanism
1. Request Initiation	**React Client**	User submits the "Add Stock" form (e.g., 50 units, Warehouse A, Reason: New Shipment).	Client-side validation for non-negative input.
2. Routing & Security	**Express Route**	The request hits POST /stock/add/:productId. The fetchuser middleware verifies the JWT and attaches req.userId.	**JWT Verification** and **User Authentication**.
3. Controller Orchestration	**Stock Controller**	Calls the Mongoose Model method: Product.findById(productId).exec(product => product.addStock(...))	Checks Product ownership.
4. Core Business Logic	**Products Model**	The addStock instance method performs two synchronous actions: updates **locationStock** quantity and pushes a new entry to **stockMovements**.	Single Document Update Guarantee.
5. Middleware	**Mongoose**	Runs the	**Atomic Data**

Trigger	**Pre-save Hook**	calculateStockAndAlerts() function before save.	**Consistency** check.
5a. Global Recalculation	**Middleware**	Iterates over the locationStock array to recalculate and update the top-level **totalStock** field.	Ensures totalStock always matches the sum of all locations.
5b. Low Stock Check	**Middleware**	Compares totalStock against globalMinStock. If low, creates a new **StockAlert** document in the separate collection.	**Automated Alert Generation**.
6. Database Write	**MongoDB**	The single, updated Products document is persisted to the database.	Transactional write operation (on one document).
7. Final Response	**Express Controller**	Returns the HTTP 200 status and the fully updated Product JSON object to the client.	Triggers client-side state refresh.

5. LOW-LEVEL DESIGN (Pages 9-17)

The low-level design details the structure and internal logic of the Express backend and the React frontend, showcasing the application of the MVC pattern.



5.1 Backend Component Breakdown and Flow

The Node.js/Express backend follows a strict folder structure:

```
/backend
├── /config
│   ├── db.js (MongoDB connection)
├── /middleware
│   ├── fetchuser.js (JWT authentication)
│   ├── validation.js (Express-validator schemas)
├── /models
│   ├── User.js
│   ├── Products.js (Most complex model)
│   ├── Location.js
│   ├── StockAlert.js
├── /routes
│   ├── auth.js (User creation, login, profile)
│   ├── product.js (Product CRUD)
│   ├── stock.js (Stock Operations: add, remove, transfer)
```

```
├── server.js
```

A. Security Middleware (middleware/fetchuser.js)

This middleware is applied to every protected route to enforce NFR-SEC-001 and NFR-SEC-002.

Logic:

1. Check for the presence of the **authtoken** in the request headers. If missing, return **401 Unauthorized**.
2. Use jsonwebtoken.verify() to validate the token.
3. Attach the user ID to the request object: req.userId = data.user.id;.
4. Call next() to pass control to the Controller.

B. Controller Logic (Example: stock.js/addStockController)

The controller is lean, focusing on input processing and calling the business layer (Model).

Pseudo-Code Flow:

```
// Function: addStockController(req, res)
const addStockController = async (req, res) => {
  try {
    // 1. Get validated inputs and userId from request
    // ... validation and extraction ...
    const userId = req.userId;

    // 2. Find the product and check authorization (NFR-SEC-002)
    let product = await Product.findOne({ _id: productId, createdBy: userId });
    if (!product) { return res.status(404).send("Product not found or unauthorized"); }

    // 3. Execute core business logic via Model instance method
    const updatedProduct = await product.addStock(locationId, quantity, userId, reason,
referenceId);

    // 4. Respond
    res.json({ success: true, product: updatedProduct });

  } catch (error) {
    // ... error handling ...
    res.status(500).send("Internal Server Error");
  }
};
```

C. Model Instance Method (Example: Products.js/addStock)

This enforces reliability (NFR-REL-006) by managing the internal integrity of the Products document.

Flow within product.addStock(...):

1. **Input Check:** Validate quantity and locationId.
2. **Location Search:** Find or create the object in the product.locationStock array.
3. **Update Quantity:** Increment the quantity.
4. **Create Audit Log:** Push the movement object to product.stockMovements.
5. **Save:** await product.save(). This triggers the **Mongoose Pre-save Middleware** for total stock recalculation.

5.2 Frontend Component Breakdown (React)

The React frontend utilizes the Context API for authentication and component-level state for data fetched from the API.

A. State Management and Data Fetching Strategy

- **AuthContext.js:** Stores authToken and userProfile.
- **Data Fetching:** All component data is fetched using useEffect hooks and **Axios**. The strategy involves re-fetching product data after any stock operation to ensure the dashboard and detail views are updated in real-time.

B. Component Hierarchy for Detailed Stock Management (StockManagement.js)

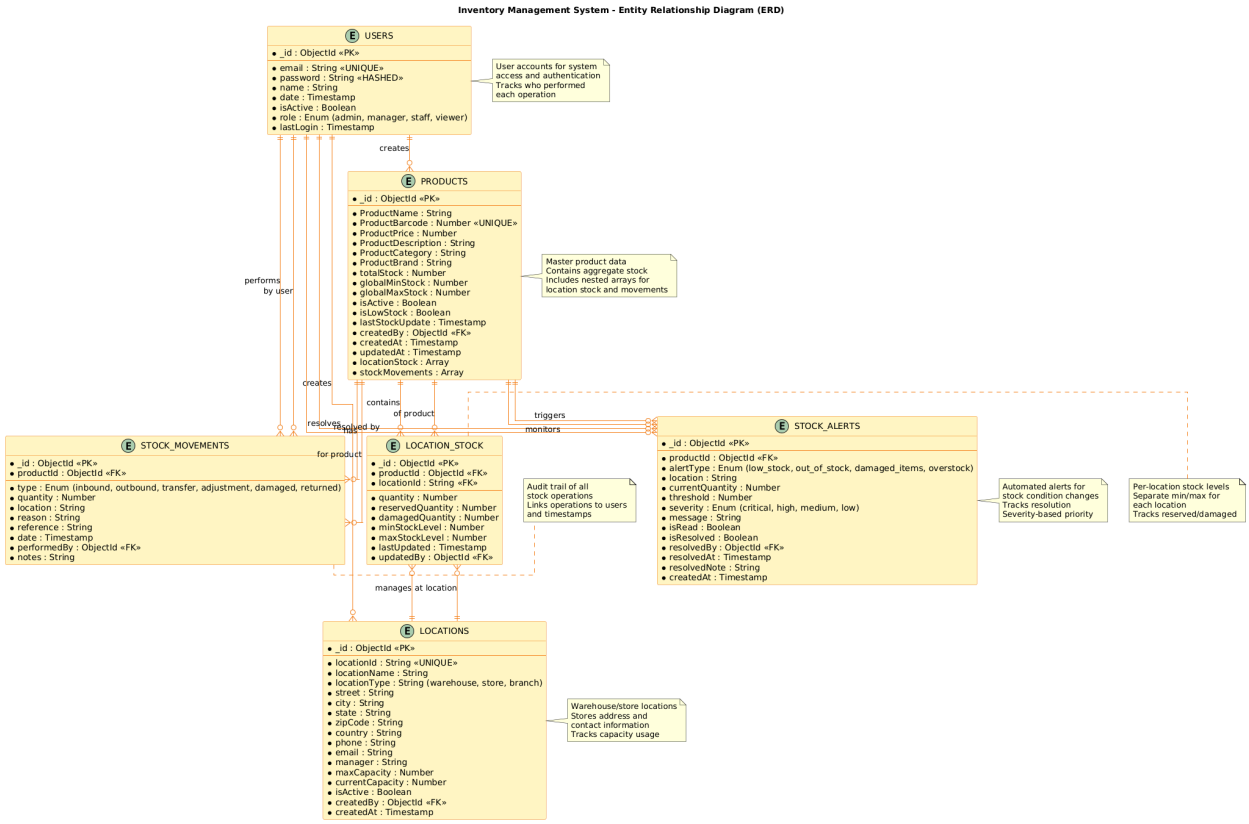
This page demonstrates complex component orchestration for user tasks:

```
graph TD
  A[StockManagement.js (Parent Page)] --> B(StockDetailsHeader);
  A --> C(LocationStockTable);
  A --> D(StockOperationTabs);
  D --> E[AddStockForm];
  D --> F[RemoveStockForm];
  D --> G[TransferStockForm];
  A --> H(StockMovementHistory);

  subgraph Data Flow
    A -- passes product.locationStock --> C
    A -- passes product ID & API call function --> D
  end
```

6. DATABASE DESIGN (Pages 18-19)

The IMS database is modeled in MongoDB using Mongoose, balancing normalization (for master data) and embedding (for performance and auditability).



6.1 Data Modeling Principles Revisited

1. **Embedding for Audit Trails:** By embedding stockMovements and locationStock within the Products document, we ensure that every stock update is a single, atomic operation (NFR-REL-006).
2. **Indexing Strategy (NFR-PERF-004):** Unique Index on **Products.ProductBarcode**. Compound Index on **{ createdBy: 1, totalStock: 1 }** for low-stock queries.

6.2 Key Collections and Mongoose Schemas (Pseudo-Code)

A. PRODUCTS MODEL (Products.js)

Includes master data, authorization, core stock fields (totalStock, globalMinStock), and embedded arrays.

```
const ProductSchema = new mongoose.Schema({
  ProductName: { type: String, required: true },
  ProductBarcode: { type: String, required: true, unique: true },
  totalStock: { type: Number, default: 0, index: true },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true, index: true
},
  locationStock: [LocationStockSchema],
  stockMovements: [StockMovementSchema]
});
```

B. LOCATION STOCK SUB-SCHEMA (Embedded in Products.js)

Tracks inventory by physical storage location.

```
const LocationStockSchema = new mongoose.Schema({
  location: { type: String, required: true },
  quantity: { type: Number, default: 0 },
  minStockLevel: { type: Number, default: 5 },
});
```

C. STOCK MOVEMENT SUB-SCHEMA (Embedded in Products.js)

The non-mutable audit log (FR-AUDIT-010).

```
const StockMovementSchema = new mongoose.Schema({
  type: { type: String, enum: ['inbound', 'outbound', 'transfer', 'adjustment'], required: true },
  quantity: { type: Number, required: true },
  location: { type: String, required: true },
```

```
    reason: { type: String, required: true },
    performedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
});
```

D. STOCK ALERT MODEL (StockAlert.js)

A separate collection used for centralized alert management (FR-ALERT-008).

```
const StockAlertSchema = new mongoose.Schema({
    product: { type: mongoose.Schema.Types.ObjectId, ref: 'Products', required: true, index:
true },
    alertType: { type: String, enum: ['low_stock', 'out_of_stock', 'damaged_excess'], required:
true },
    severity: { type: String, enum: ['Low', 'Medium', 'High', 'Critical'], default: 'Medium', index:
true },
    isResolved: { type: Boolean, default: false, index: true },
});
```

7. NETWORK SYSTEM DESIGN (Pages 20-22)

The production environment is designed for **High Availability (HA)** and **Scalability (NFR-SCAL-005)** using a cloud-native approach.

7.1 Scalability Architecture

The system uses horizontal scaling across both the API and Data layers.

- **Frontend (React Static Assets):** Hosted on a **Content Delivery Network (CDN)** to minimize latency globally.
- **API Gateway (Express Cluster):**
  1. Multiple Node.js instances run on an **Auto-Scaling Group (ASG)** managed by **PM2**.
  2. An **Elastic Load Balancer (ELB)** distributes incoming traffic across the instances and handles SSL termination.
  3. The API is **Stateless** (using JWT), allowing simple horizontal scaling.

7.2 Database Cluster Configuration

For data redundancy and read/write separation:

- **Replica Set:** A minimum three-member replica set (1 Primary, 2 Secondary) is deployed.
  - **Primary Node:** Handles all write operations (stock updates).
  - **Secondary Nodes:** Handle read-heavy operations, such as generating reports and dashboard data, distributing the read load.
- **Data Partitioning (Future):** Sharding, based on the createdBy field, is planned for managing future massive data growth.

7.3 Infrastructure Tools

Tool/Technology	Purpose in IMS Deployment	Benefits
Docker & Kubernetes	Containerization and orchestration of the Node.js/Express application.	Provides environment consistency, automated deployment rollouts, and self-healing.
PM2	Process Manager for Node.js.	Maintains process state and automatically restarts API instances upon code crash.
Nginx (or ELB)	Reverse Proxy and Load Balancer.	Distributes traffic evenly and handles SSL.

8. FUTURE IMPROVEMENTS (Pages 23-25)

The roadmap focuses on transforming the IMS from a tracking system into a full-fledged business planning component.

8.1 Short-term Improvements (Focus on Usability & Efficiency)

1. **Barcode/QR Scanning:** Implement a dedicated React component to use the device camera API for faster **Stock Inbound** and **Outbound** operations.
2. **Excel/CSV Import/Export:** Add bulk data handling capabilities for new product masters and detailed inventory report generation.
3. **Advanced UI/UX Refinement:** Utilize advanced grid libraries (e.g., TanStack Table) for deep client-side filtering, sorting, and better data visualization.

8.2 Mid-term Improvements (Focus on Workflow Automation and Control)

1. **Vendor and Purchasing Module:** Introduce dedicated Vendor and PurchaseOrder master models to formally integrate **Stock Inbound** operations with PO tracking.
2. **Role-Based Access Control (RBAC):** Implement granular authorization levels (Admin, Manager, Stock Clerk) by updating the fetchuser middleware and adding role checks to all controller functions.
3. **Asynchronous Processing:** Use a dedicated task queue (e.g., Redis) to handle background tasks such as batch data imports and sending email notifications for critical alerts, maintaining low latency for synchronous user interactions.

8.3 Long-term Enhancements (Focus on System Resilience and Business Intelligence)

1. **Advanced Monitoring & Observability:** Implement a full **ELK Stack** for centralized log analysis and integrate **Prometheus/Grafana** for metrics and visualization, tracking key metrics like **API Latency (p95)** and **Database Query Duration**.
2. **Automated Testing Suite:** Implement comprehensive **Unit Tests** (Mocha/Chai) for Model methods and **Integration Tests** (Supertest) for all API routes to ensure robustness and prevent regressions.
3. **Containerization Finalization:** Transition the application into a fully managed Kubernetes cluster for robust deployment, utilizing features like rolling updates and automated rollbacks.